

Route to Clean Cities: Waste Collection Optimization with UGVs

Farah Ahmed¹, Hoor Amr¹, Marwan Khalil², Maryam Mohamed², Youssef Amir², Ziad Yasser²,
Jessica Magdy^{1,3}, and Omar M. Shehata^{1,3}

¹*Mechatronics Engineering Dept., Faculty of Engineering and Materials Science, German University in Cairo*

²*Computer Science Dept., Faculty of Media Engineering and Technology, German University in Cairo*

³*Multi-Robot Systems (MRS) Research Group, Cairo*

{farah.yousef, hoor.awad, marwan.ahmed, maryam.alsayed, youssef.elhusseiny,
ziad.gafar}@student.guc.edu.eg,
{jessica.gergis, omar.mohamad}@guc.edu.eg

Abstract—Efficient waste collection is crucial to effective urban management, with far-reaching implications for environmental sustainability, public health, and overall quality of life in cities. As urban populations continue to grow, the demand for optimized waste management systems becomes increasingly critical. This study addresses the challenge of designing efficient waste collection routes using metaheuristic optimization techniques. Specifically, four different algorithms are employed to minimize route lengths, reduce operational costs, and improve the scalability of waste collection systems. These algorithms are rigorously evaluated in a variety of test configurations that simulate real-world scenarios. The results of the experiments reveal the strengths and limitations of each algorithm, providing valuable insights into their applicability for real-world waste management systems.

Index Terms—waste collection, route optimization, metaheuristic algorithms, SA, GA, DPSO, AIS

I. INTRODUCTION

Waste collection is a complex logistical challenge with significant environmental and economic implications as mentioned in [1] and [2]. Inefficient systems raise costs and emissions, necessitating advanced optimization techniques. This study aims to evaluate and compare the performance of four optimization algorithms—Simulated Annealing (SA), Genetic Algorithm (GA), Discrete Particle Swarm Optimization (DPSO), and Artificial Immune System (AIS) in optimizing waste collection routes. The objective is to assess their efficiency, scalability, and adaptability in order to reduce costs, minimize emissions, and improve overall system performance in real-world waste collection scenarios, through running different test cases,

II. LITERATURE REVIEW

Optimization algorithms are widely used to address complex problems, including waste management in smart cities. The following literature highlights some important key studies and their contributions. In some cases, they tackled the same

problem as this research and in others have applied the same algorithms used but on different topics.

A paper study uses a genetic optimization algorithm to adequately plan vehicle routes for collecting municipal waste using real data. The effectiveness of the genetic algorithm has been tested with that of another highly effective algorithm which is the Variable Neighborhood Search, to ensure that the result is efficient [1]. In another study, the authors propose a hybrid methodology that combines K-Means clustering and genetic algorithms to improve the efficiency of waste collection routes in Ballari City, Karnataka, India [3]. A different study evaluates metaheuristic methods such as particle swarm optimization (PSO), ant colony optimization (ACO), artificial bee colony (ABC), genetic algorithm (GA), simulated annealing (SA) and Tabu search (TS) for waste collection. The metaheuristic approaches are flexible in selecting appropriate goals and constraints; however, they have some shortcomings in terms of parameter setting and computation complexity [4].

A different study compares Genetic Algorithms (GA) and Particle Swarm Optimization (PSO) for job-shop scheduling, where GA explores large search spaces but converges slowly, while PSO is faster but prone to local optima. Hybrid approaches that combine these methods improve performance, evaluated using many metrics [5].

Another study explores IoT-based waste management optimization, achieving cost savings and routing efficiency, but faces challenges in data accuracy [6] and a varying study focuses on IoT-assisted vehicle routing, where real-time analytics improve efficiency, but system integration remains a challenge [2].

Paper [7] develops an intelligent urban waste removal system using ROS by automatic communication between individual waste containers and a waste collection vehicle where the bin must be emptied, while an alternative study uses repeated neural networks (RNNs) in AI-driven waste predic-

tion, improving accuracy but struggling with real-world complexities [8].

A different study reduces power losses using an artificial immune system optimization technique in electrical distribution networks [9].

III. PROBLEM FORMULATION

The goal is to optimize the routes for unmanned vehicles to collect waste, balancing maximum waste collection and minimal fuel consumption. Vehicles follow routes between nodes, collect waste and return to a dump site, represented in this paper as the first column of the map. Where the map is a designed environment that's scalable so as the number of vehicles increase the map gets bigger and vice versa to allow for different cases (altered variables). Each vehicle operates under capacity, time, and collision constraints.

A. Composite Objective Function

The composite objective function balances minimizing fuel consumption and maximizing waste collection. For a fleet of m vehicles:

$$\min \left(\sum_{i=1}^m \sum_{j=1}^{T_i-1} f \cdot d(p_{ij}, p_{ij+1}) + \alpha \cdot \left(\frac{W_{\text{remaining}}}{W_{\text{total}}} \right) \right) \quad (1)$$

where f is the fuel consumed, $d(p_{ij}, p_{ij+1})$ is the distance between consecutive nodes in path P_i , $W_{\text{remaining}}$ is the remaining waste to be collected, W_{total} is the total waste in the map, and α is a trade-off factor.

B. Decision Variables

The decision variable is a set of paths $\mathcal{P} = \{P_1, P_2, \dots, P_m\}$, where each P_i represents the sequence of nodes visited by vehicle i , starting from the parking slot and ending at the dump site.

C. Constraints

1) *Capacity Constraint*: Waste collected by each vehicle must not exceed its capacity:

$$\sum_{i=1}^m W_i \leq \text{maxCapacity}.$$

2) *Waste Collection Requirement*: Vehicles must collect waste before returning to the dump site: $W_i > 0 \quad \forall i$.

3) *No Collisions*: No two vehicles can occupy the same node or cross paths at the same time.

D. Assumptions

- Uniform fuel consumption per kilometer.
- Randomized waste intensity and collection time.
- Equal maximum waste capacity for all vehicles.

E. Solution Representation

The solution is represented as a matrix of paths, where each row corresponds to a vehicle and each column represents its position at a specific time step:

$$\text{Sol} = \begin{bmatrix} p_{1,1} & p_{1,2} & \dots & p_{1,n} \\ p_{2,1} & p_{2,2} & \dots & p_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ p_{m,1} & p_{m,2} & \dots & p_{m,n} \end{bmatrix}$$

Here, m is the number of vehicles, n is the number of time steps, and $p_{i,j}$ represents vehicle i 's position at time step j .

IV. OPTIMIZATION TECHNIQUES

Four different optimization techniques were applied to compare and contrast between computational complexity, repeatability, and optimality by the help of graphs following the fitness value calculated.

A. Simulated Annealing (SA)

Simulated Annealing optimizes vehicle routing by iteratively improving solutions while occasionally accepting worse solutions to escape local optima. The process includes:

1) *Initialization*: Set parameters like initial and final temperatures (T_{init} , T_{final}) to indicate when the code terminates (as temperature decreases the code converges), cooling coefficient (α), iterations per temperature (n_T), and problem-specific variables (map, destinations, vehicle capacities).

2) *Generate New Solutions*: Randomly swap destinations to create a feasible solution.

3) *Energy Change (ΔE)*: Compute the difference between the current and new solution's objective function values.

4) *Solution Evaluation*: Accept better solutions directly; however, for worse solutions, calculate probability $p = e^{-\Delta E/T}$ and accept if $p > r$, where r is a random number between 0 and 1.

5) *Update Best Solution*: Save the best solution encountered.

6) *Temperature Reduction*: Decrease temperature using the geometric cooling schedule to gradually decrease the likelihood of accepting worse solutions.

7) *Termination*: Stop when $T \leq T_{\text{final}}$ or the maximum number of iterations is reached.

B. Genetic Algorithm (GA)

Genetic Algorithm optimizes vehicle routing by evolving solutions through selection, crossover, and mutation. The process involves:

1) *Initialization*: Set GA parameters such as iterations ($Iterations_{no.}$), number of vehicles ($Vehicles_{no.}$), population size (m), and percentages for elitism ($Elitism\%$), crossover ($CrossOver\%$), and mutation ($Mutation\%$). Define map, destinations, and vehicle parking slots.

2) *Generate Initial Population*: Create m random feasible solutions for each vehicle, where each solution represents a route configuration.

3) *Evaluate Fitness*: Calculate fitness values for all solutions to assess alignment with objectives.

4) *Generate New Solutions*: using three genetic operators: elitism, crossover, and mutation. Elitism retains the top-performing solutions for the next generation. Crossover, using Davis' Order Crossover method, combines the 1st best solution with others (e.g., 2nd and 3rd best) to produce offspring. Mutation randomly swaps two points in a solution to maintain diversity and avoid local optima.

5) *Feasibility Check*: Ensure all new solutions meet constraints, regenerating invalid solutions if necessary.

6) *Update Best Solution*:: Track the best solution if a new one with better fitness is found.

7) *Termination*:: Continue until the maximum number of generations or stopping criteria (e.g., no fitness improvement) is reached.

C. Discrete Particle Swarm Optimization (DPSO)

Discrete Particle Swarm Optimization optimizes vehicle routing by updating particle positions and velocities in a discrete solution space. The process includes:

1) *Initialization*: set DPSO parameters such as max no. of iterations, number of vehicles, swarm size, inertia weight (w), and acceleration coefficients (c_1, c_2). Generate an initial swarm of particles representing feasible vehicle routes, evaluate their fitness, and initialize personal best (P_{best}) and global best (G_{best}) solutions.

2) *Velocity Update*: Update each particle's velocity to get the sense of its direction using the following equation:

$$v_i^{t+1} = w \cdot v_i^t + c_1 \cdot r_1 \cdot (P_{best} - x_i^t) + c_2 \cdot r_2 \cdot (G_{best} - x_i^t)$$

where w controls exploration, c_1 and c_2 weight personal and global best influences, and r_1, r_2 are random numbers between 0 and 1.

3) *Position Update*: Update each particle's position in the solution space by adding the velocity vector to the current position using the following equation:

$$x_i^{t+1} = x_i^t + v_i^{t+1}$$

Convert continuous position vectors to discrete solutions using the Great Value Priority (GVP) rule.

4) *Fitness Evaluation*: Calculate the fitness value of each particle based on the objective function. Update P_{best} and G_{best} if better solutions are found.

5) *Process Continuation*: Repeat the velocity and position updates until termination criteria is met.

6) *Termination*: Conclude the process when the maximum iterations or stopping criteria are satisfied. The best solution found is returned as the final result.

D. Artificial Immune System (AIS)

Artificial Immune System Optimization (AIS) models immune system processes to solve vehicle routing problems. The algorithm involves:

1) *Initialization*: Define AIS parameters such as population size (N), number of generations (Gen), mutation rate (MR), and clonal rate (CR). Generate an initial population of antibodies, each representing a feasible vehicle routing solution.

2) *Fitness Evaluation*: Calculate the fitness of each antibody based on the objective function, which includes the distance traveled, the efficiency of waste collection, and the restrictions.

3) *Cloning and Mutation*: Clone the top-performing antibodies based on CR and apply mutations to clones inversely proportional to their fitness, modifying vehicle routes to explore new solutions.

4) *Selection*: Select the fittest antibodies from the cloned and mutated population to form the next generation.

5) *Diversity Maintenance*: Introduce new random antibodies to maintain diversity and prevent premature convergence.

6) *Update Best Solution*: Track the best solution encountered during the optimization process.

7) *Termination*: Stop when the maximum number of generations (Gen) is reached or no significant improvement is observed in fitness. Output the best solution as the final result.

V. RESULTS

To evaluate the performance of the optimization algorithms, we conducted tests. The testing process involves running each algorithm with specific parameters and analyzing the results, including visualizations of vehicle routes, in the form of a map with each path color representing a vehicle and fitness evolution over iterations.

Different test cases were designed to evaluate their performance of the four optimization techniques.

A. SA Results

1) *Test Case 1 (Base Case)*: This base case serves as the reference for all comparisons. **Parameters:**

- $\alpha = 0.7$
- $T_{\text{init}} = 1000$
- $T_{\text{final}} = 20$
- $n_T = 5$
- n (vehicles no.) = 4

2) *Test Case 2*: This test demonstrates the scalability of the algorithm to larger fleets. **Parameters altered from base case:**

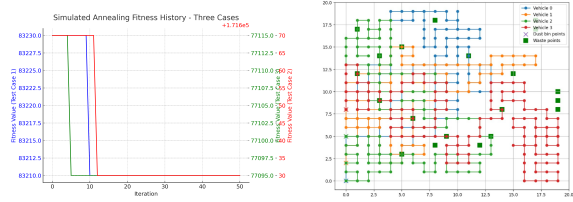
- n (number of vehicles) = 6

3) *Test Case 3*: In this case, the SA parameters were modified by increasing the final temperature and α . These changes aim to reduce computation time, though potentially at the cost of accuracy. **Parameters altered from base case:**

- $\alpha = 0.8$
- $T_{\text{final}} = 200$

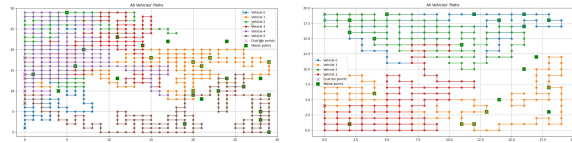
4) *Summary of Results:*

- **Test Case 1 (Base Case):** Served as the benchmark with well-optimized vehicle paths. Distinct vehicle zones with minimal overlap ensured efficient coverage. Fitness history showed smooth convergence with minor fluctuations.
- **Test Case 2:** Moderate optimization with some inefficiencies. Vehicles showed increased congestion, particularly in overlapping zones for Vehicles 0, 1, and 5. Noticeable fitness fluctuations indicate the need for parameter tuning to achieve smoother convergence.
- **Test Case 3:** Poor optimization with inefficient vehicle paths. Significant path overlaps and redundant travel were observed across multiple vehicles. The fitness history showed a rapid decline, highlighting lower fitness yet the need for better tuning.



(a) Test 1, 2, & 3 fitness graphs combined

(b) Test 1 map



(c) Test 2 map

(d) Test 3 map

Fig. 1: Vehicle paths for different test cases

B. GA Results

1) *Test Case 1 (Base Case)*: This base case serves as the reference for all comparisons. **Parameters:**

- $\text{Mutation}_{\%} = 20\%$
- $\text{Elitism}_{\%} = 20\%$
- $\text{CrossOver}_{\%} = 60\%$
- $\text{Population}_{\text{size}}(m) = 5$
- $\text{Vehicles}_{\text{no.}} = 4$
- $\text{Iteration}_{\text{no.}} = 100$

2) *Test Case 2*: This test demonstrates the scalability of the algorithm to a larger fleet. **Parameters altered from base case:**

- $\text{Vehicles}_{\text{no.}} = 6$

3) *Test Case 3*: This case tests the performance with a moderately larger fleet size while keeping other parameters constant. **Parameters altered from base case:**

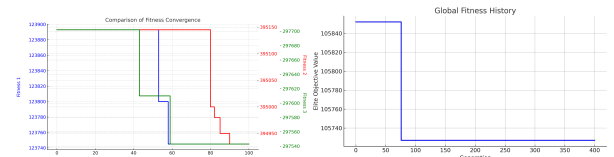
- $\text{Vehicles}_{\text{no.}} = 5$

4) *Test Case 4*: This case evaluates the impact of increased iterations, exploring the algorithm's convergence behavior over a prolonged runtime. **Parameters altered from base case:**

- $\text{Iterations}_{\text{no.}} = 400$

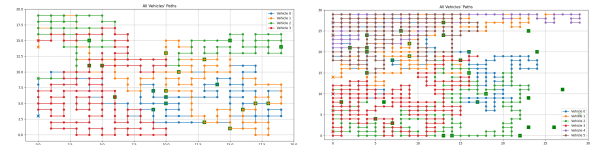
5) *Summary of Results:*

- **Test Case 1 (Base Case):** Served as the benchmark with well-optimized vehicle paths. Vehicles covered distinct zones with minimal overlap, ensuring efficient coverage. Fitness history showed a smooth downward trend, indicating strong optimization.
- **Test Case 2:** Higher final fitness values compared to the base case, indicating increased inefficiencies. Notable overlap in vehicle paths, particularly for Vehicles 0, 1, and 4, led to congestion. The decline in fitness values was slower than in the base case, highlighting less effective optimization.
- **Test Case 3:** Moderate inefficiencies with final fitness values higher than the base case but lower than Test Case 2. Overlap in paths, particularly between Vehicles 0, 2, and 3, reduced optimization effectiveness. Convergence was less stable than the base case, showing fluctuations but still trending downward.
- **Test Case 4:** Similar to the base case with comparable final fitness values. Vehicles maintained distinct zones, minimizing overlap and redundant paths. Extended iterations had little additional impact on optimization having a slightly less fitness.



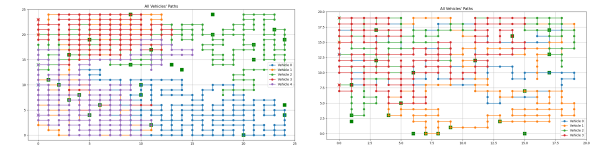
(a) Test 1, 2, & 3 fitness graphs combined

(b) Test 4 fitness graph



(c) Test 1 map

(d) Test 2 map



(e) Test 3 map

(f) Test 4 map

Fig. 2: Fitness graphs and vehicles' maps

C. DPSO Results

Unlike SA and GA, DPSO calculates a single fitness value per map rather than per vehicle.

1) *Test Case 1 (Base Case)*: This base case tests DPSO for a moderate fleet size with standard parameters. **Parameters:**

- $Vehicles_{no.} = 4$
- $Swarm_{size} = 5$
- $Iterations_{no.} = 100$
- $w = 0.792$
- $c_1 = c_2 = 1.4944$

2) *Test Case 2*: This case evaluates the effect of increased iterations, examining whether prolonged optimization improves convergence and reduces the objective value. **Parameters altered from base case:**

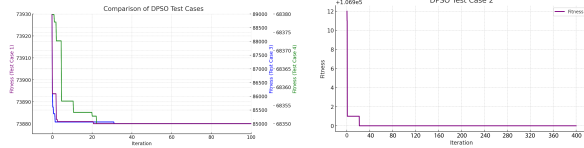
- $Iterations_{no.} = 400$

3) *Test Case 3*: This case tests the impact of a decreased inertia weight, exploring its effect on the balance between exploration and exploitation. **Parameters altered from base case:**

- $w = 0.5$

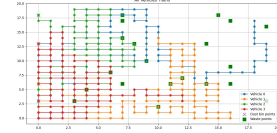
4) *Test Case 4*: This test demonstrates the scalability of DPSO for a larger fleet, evaluating its performance with an increased number of vehicles. **Parameters altered from base case:**

- $Vehicles_{no.} = 5$

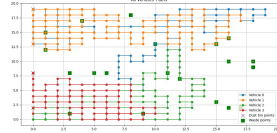


(a) Test 1, 3, & 4 fitness

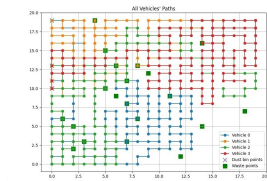
(b) Test 2 fitness graph



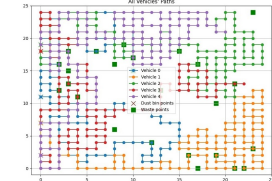
(c) Test 1 map



(d) Test 2 map



(e) Test 3 map



(f) Test 4 map

Fig. 3: Fitness graphs and vehicles' maps

5) Summary of Results:

- **Test Case 1 (Base Case)**: Benchmarked performance with stable convergence and effective optimization, though some path overlap resulted in minor inefficiencies.
- **Test Case 2**: Higher fitness values than the base case with increased inefficiencies due to path redundancy and less distinct map partitioning. Extended iterations did not significantly improve results, suggesting the need for parameter tuning.
- **Test Case 3**: Reduced inertia weight led to higher fitness values and lower optimization

quality compared to the base case. Map partitioning was less distinct, and convergence was rapid but resulted in suboptimal solutions.

- **Test Case 4**: Significant improvements in route optimization and task distribution. Vehicle paths exhibited reduced overlap and better task allocation. Convergence was slower but more effective, resulting in lower fitness values and enhanced fleet utilization compared to the base case.

D. AIS Results

Like DPSO, AIS calculates a single fitness value per map rather than per vehicle.

1) *Test Case 1 (Base Case)*: This base case tests AIS for a moderate fleet size with standard parameters. **Parameters:**

- $Clones_{no.} = 50$
- $Rate_{mutation} = 1.0$
- $Generations_{no.} = 100$
- $Population_{size} = 50$
- $Vehicles_{no.} = 4$

2) *Test Case 2*: This test evaluates the performance with a larger number of clones. **Parameters altered from base case:**

- $Clones_{no.} = 100$

3) *Test Case 3*: This test demonstrates the effect of a decreased mutation rate. **Parameters altered from base case:**

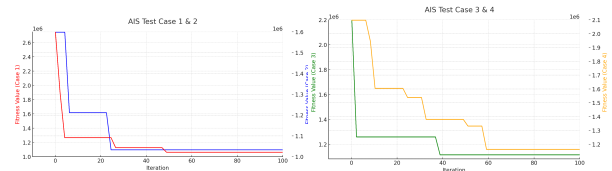
- $Rate_{mutation} = 0.3$

4) *Test Case 4*: This test evaluates the scalability of the algorithm to a larger fleet. **Parameters altered from base case:**

- $Vehicles_{no.} = 6$

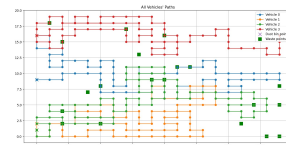
5) Summary of Results:

- **Test Case 1 (Base Case)**: Stable convergence with moderate execution time and efficient optimization for a standard fleet size.
- **Test Case 2**: Increasing the number of clones improved solution diversity but slightly increased execution time without significant improvement in final fitness.
- **Test Case 3**: Decreasing the mutation rate reduced computational time, but convergence was less effective, leading to suboptimal paths.
- **Test Case 4**: Scalability demonstrated with a larger fleet size, achieving effective optimization, though execution time increased due to additional vehicles.

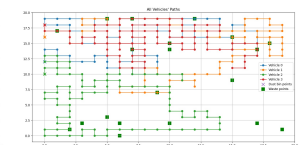


(a) Test 1, & 2 fitness graphs combined

(b) Test 3, & 4 fitness graphs combined



(c) Test 1 map



(d) Test 2 map

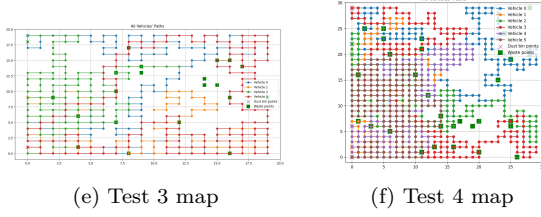


Fig. 4: Fitness graphs and vehicles' maps

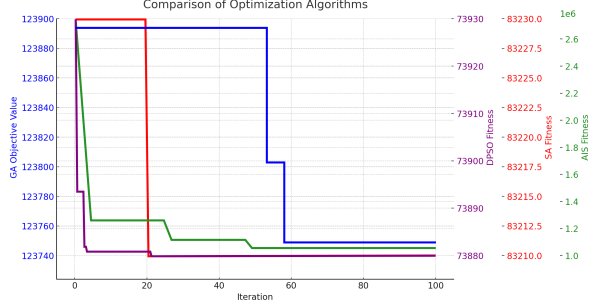


Fig. 5: Comparison of convergence rates for different algorithms over iterations.

E. Observations

The algorithms were run 10 times each all on the fixed base case and gave more insight into the process of each algorithm. The average, minimum and standard deviation, of the least fitness within all runs, were calculated to determine the features of each algorithm. Such features include, convergence rate (CR), computational time (CT), optimality and repeatability. The convergence rate it's the iteration at which the algorithm found the best solution that was displayed at the end, an average of all runs was obtained. The computational time is the time elapsed between the start of the code and the obtaining of the final solution. As for the optimality was determined as the best fitness achieved within those 10 runs. Lastly the repeatability is determined through the standard deviation (SD), the less the more repeatable.

TABLE I: Comparison of Optimization Algorithms

Algorithm	SA	GA	DPSO	AIS
CR	Average Iterations			
	23	59	24	50
CT	Average Seconds			
	18	28	729	212
Optimality	Fitness			
	48933	102125	2470	38000
Average	Fitness			
	68888	140706	26559	545040
SD	Fitness			
	4043	9164	8499	158197
Repeatability	Rank			
	First	Third	Second	Fourth

Overall, the most repeatable and fastest algorithm was SA, although the second repeatable yet

slowest algorithm, DPSO, gave the most optimal solution giving the lowest average fitness as well. The rest of the calculations and statistical analysis is demonstrated in Table I. Where the best of each statistical asset is highlighted in bold.

VI. CONCLUSION

This study proposed a solution for the waste collection problem within smart cities using optimization algorithms, specifically four metaheuristic algorithm. Although SA had the fastest execution time, DPSO achieved the most optimal solutions despite being computationally expensive. GA and AIS showed varying efficiency, with GA being moderately fast but less optimal, and AIS demonstrating scalability but with higher computation time. Future work could explore hybrid approaches that combine the strengths of these algorithms to further enhance waste collection efficiency.

REFERENCES

- [1] E. Alberdi, L. Urrutia, A. Goti, and A. Oyarrbide-Zubillaga, "Modeling the Municipal Waste Collection Using Genetic Algorithms," *Processes*, vol. 8, no. 5, p. 513, 2020.
- [2] S. Sadiq, R. Khusainov, and E. Akhmetshin, "Internet of Things-assisted vehicle route optimization for municipal solid waste collection," *Applied Sciences*, vol. 14, no. 1, p. 287, 2024.
- [3] A. Sharma, R. Patel, and S. Nair, "GreenRoute Ballari: A Hybrid Approach for Optimizing Waste Collection Routes Using K-Means Clustering and Genetic Algorithm," *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, 2024.
- [4] N. L. Rachmawati, Y. A. Iskandar, D. P. Putri, and M. Lusiani, "A sustainable supply chain management model for cleaner production performance using optimization algorithms," *Journal of Cleaner Production*, vol. 123557, 2020.
- [5] N. L. Rachmawati, Y. A. Iskandar, D. P. Putri, and M. Lusiani, "Optimization of genetic and particle swarm algorithms for scheduling and supply chain problems," in *Proceedings of the 2023 IEOM International Conference*, Manila, Philippines, 2023.
- [6] M. Al Jubair, M. Alhaffar, and S. AlMasri, "Optimal management of solid waste in smart cities using Internet of Things," *ResearchGate*, 2019. [Online].
- [7] A. Kumar, R. Gupta, and S. Verma, "An Intelligent Waste Removal System for Smarter Communities," in *Proceedings of the International Conference on Smart Cities and Environmental Sustainability*, 2020.
- [8] E. Santiago, A. Figueiredo, and J. Ramos, "Optimization of waste collection in smart cities with the use of evolutionary algorithms," *ResearchGate*, 2020.
- [9] M. A. Abido and Y. M. Abu-Mouti, "Artificial Immune Systems Optimization Approach for Multiobjective Distribution Feeder Reconfiguration," in *Proceedings of the 2014 IEEE Congress on Evolutionary Computation (CEC)*, Beijing, China, 2014.